

Mod 4

Due on April 17, 2025

Tuesday/Thursday 11:00-12:15, Warner 209

Mark Floryan - Section 001

Rushil Umaretiya

frj2ka@virginia.edu

#1 Give implementation level descriptions for Turing Machines that decides the following two languages.

- $\{w \mid w \text{ contains an equal number of 0s and 1s}\}$
 1. Start with a counter at the tape head initialized to 0.
 2. Scan the input from left to right.
 - (a) If you see a 0, increment the counter by 1.
 - (b) If you see a 1, decrement the counter by 1.
 - (c) If you see any other symbol, reject the input.
 3. If the counter is 0 at the end of the scan, accept the input.
 4. If the counter is not 0 at the end of the scan, reject the input.
- $\{W = w\#w \mid w \in \{0,1\}^*\}$
 1. Scan the input from left to right to ensure that the input only contains one $\#$ along with exclusively 1's and 0's and return to the beginning of the input.
 2. Write a $\$$ to the tape and move the head to the right.
 3. For each symbol on the input that is not the $\#$, write the same symbol to the tape and move the head to the right.
 4. When the input reaches the $\#$, write $\#$ to the tape and move the head left until it reaches $\$$ of the input.
 5. Move the head to the right and compare the next symbol on the tape with the symbol on the input.
 - (a) If the symbols match, move the head to the right and repeat step 5.
 - (b) If the symbols do not match, reject the input.
 - (c) If the symbol on the tape is $\#$ and there is nothing left on the input, accept the input.

#2 Let $M_{DFA} = \{A \mid A \text{ is a DFA that does not accept any string with an odd number of 1's}\}$. Show that M_{DFA} is decidable.

Proof. Since DFA's always terminate and are closed under intersection, we can construct a DFA that accepts the language $B = \{w \mid w \text{ has an odd number of 1's}\}$. We can then construct a new DFA M that accepts the language $L = A \cap B$.

Then we can check if L is empty by checking if the start state of M is reachable from any of the accepting states. If it is reachable, then A accepts a string with an odd number of 1's and we reject. If it is not reachable, then A does not accept a string with an odd number of 1's and we accept.

Since this algorithm always terminates, we can conclude that M_{DFA} is decidable. □

#3 Suppose we take the Turing Machine as defined in class, and we change the transition function to only allow the following: $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{S, R\}$. In other words, these machines can move the head right or stay in the same position but can never move left. Does this recognize the same set of languages as the normal *Turing Machine*. If so, why? If not, what class of languages does it recognize?

It seems that this machine can only recognize regular languages. This is because the machine can only move right and cannot move left. Therefore, it cannot keep track of the number of symbols on the tape. First we can prove that the no-left Turing machine is not as powerful as a regular TM by showing that it cannot recognize the language $L = \{a^n b^n | n \geq 0\}$.

Proof. Suppose we have a no-left Turing machine M that recognizes the language L . We can construct a string $w = a^n b^n$ and run it through the machine. The machine will start at the leftmost symbol and move right until it reaches the first b . At this point, the machine cannot go back to the left to check if there are any more a 's on the tape. Therefore, the machine cannot recognize the language L . □

Since we have shown that the no-left Turing machine cannot recognize the language L , we can conclude that it is not as powerful as a regular Turing machine. When questioned about the class of languages that it can recognize, we can say that it can only recognize regular languages as it technically has no read access to old memory. Therefore we can compare it to a DFA/NFA which can only recognize exactly the regular languages.

#4 A *Turing Machine with doubly-infinite tape* is an ordinary Turing Machine, except that the tape is infinitely indexed in both the left and right direction. Prove that this machine is equivalent in computational power to a traditional Turing Machine.

Proof. We can prove that the TM is equivalent to a DTM by showing that a TM can simulate a DTM and vice versa.

→ **TM to DTM :**

This direction is trivial. Every TM can be directly simulated by a DTM and just ignore the left bounds.

← **DTM to TM :**

To show that the DTM can be simulated by a TM, we can construct a proof similar to that of $|\mathbb{N}| = |\mathbb{Z}|$ by performing a mapping of all cells from the doubly infinite tape to the infinite tape.

We can first instantiate a variable on our TM to keep track of the true cell number, i , then we can remap i when we need to use it. This way, we can ensure that all cells on the doubly infinite tape are mapped to a unique cell on the TM.

- If i is positive, we can map it to $2i$.
- If i is negative, we can map it to $2|i| + 1$.

Every time we want to perform a read/write on the current cell, first we compute the mapping of i , moves the head to the mapped cell, and then perform the read/write operation.

Therefore, we can conclude that the two machines are equivalent in computational power. □

#5 For this question, you will do five separate proofs. Prove that the class of *Decidable Languages* is closed under *union*, *concatenation*, *star*, *complement*, and *intersection*.

- Union

Proof. If L_1 and L_2 are decidable languages, then there exist Turing machines M_1 and M_2 that decide them. We can construct a new Turing machine M that decides the union of the two languages as follows:

1. On input w , run M_1 on w .
2. If M_1 accepts, accept.
3. If M_1 rejects, run M_2 on w .
4. If M_2 accepts, accept. Otherwise, reject.

Since both M_1 and M_2 always halt, M will also always halt. Therefore, the union of two decidable languages is decidable. $\square$

- Concatenation

Proof. If L_1 and L_2 are decidable languages, then there exist Turing machines M_1 and M_2 that decide them. We can construct a new Turing machine M that decides the concatenation of the two languages as follows:

1. On input w , for each possible split of w into two parts x and y , run M_1 on x and M_2 on y .
2. If both machines accept, accept. If no split results in both machines accepting, reject.

Since both M_1 and M_2 always halt, M will also always halt. Therefore, the concatenation of two decidable languages is decidable. $\square$

- Star

Proof. If L is a decidable language, then there exists a Turing machine M that decides it. We can construct a new Turing machine M' that decides the star of the language as follows:

1. On input w , if w is the empty string, accept.
2. For each possible split of w into parts $x_1, x_2, \dots, x_n$, run M on each x_i .
3. If all machines accept, accept. Otherwise, reject.

Since M always halts, M' will also always halt and there is always a finite number of splits on our initial input. Therefore, the star of a decidable language is decidable. $\square$

- Complement

Proof. If L is a decidable language, then there exists a Turing machine M that decides it. We can construct a new Turing machine M' that decides the complement of the language as follows:

1. On input w , run M on w .
2. If M accepts, reject. If M rejects, accept.

Since M always halts, M' will also always halt. Therefore, the complement of a decidable language is decidable. $\square$

- Intersection

Proof. If L_1 and L_2 are decidable languages, then there exist Turing machines M_1 and M_2 that decide them. We can construct a new Turing machine M that decides the intersection of the two languages as follows:

1. On input w , run M_1 on w .
2. If M_1 accepts, run M_2 on w .
3. If both machines accept, accept. Otherwise, reject.

Since both M_1 and M_2 always halt, M will also always halt. Therefore, the intersection of two decidable languages is decidable. $\square$

#6 Prove the following claim: Let C be a language. Prove that C is Turing-recognizable if and only if a decidable language D exists such that $C = \{x | \exists y((x, y) \in D)\}$

Proof. ($\Rightarrow$) If C is Turing-recognizable, then there exists a decidable language D such that $C = \{x | \exists y((x, y) \in D)\}$.

Since C is Turing-recognizable, there exists a Turing machine M that recognizes it. Therefore we can build a decider D as follows:

$$D = \{(x, y) | y \text{ is the number of steps taken by } M \text{ on input } x\}$$

On input (x, y) , D will run M on input x for y steps. If M accepts, then D accepts (x, y) . If M rejects or halts without accepting, then D rejects (x, y) for all y . Thus, $C = \{x | \exists y((x, y) \in D)\}$.

($\Leftarrow$) If there exists a decidable language D such that $C = \{x | \exists y((x, y) \in D)\}$, then C is Turing-recognizable. Since D is decidable, there exists a Turing machine M_D that decides it. We can construct a Turing machine M_C that recognizes C as follows:

$$M_C = \{x | \exists y((x, y) \in D)\}$$

On input x , M_C will run M_D on input (x, y) for all possible values of y . If M_D accepts for any value of y , then M_C accepts. If M_D rejects for all values of y , then M_C rejects. Thus, C is Turing-recognizable. $\square$

#7 Consider the following decision problem: "Given a Turing Machine M and input w , does M ever move its head to the left three times in a row?". Show, via reduction, that this problem is *undecidable*.

Proof. First, let's assume there exists a solver R for the problem "Given a Turing Machine M and input w , does M ever move its head to the left three times in a row?" We can use this solver to solve the $A_{TM} = \{(M, w) \mid M \text{ is a TM and } M \text{ accepts } w\}$, which is known to be undecidable.

For any instance (M, w) of A_{TM} , let's construct a machine S that:

On input x (any input):

1. Run M on input w
2. If M accepts w , then:
 - (a) Move the head left
 - (b) Move the head left again
 - (c) Move the head left a third time
3. Otherwise, ensure the head never moves left three consecutive times (e.g., by always moving right after at most two left moves)

Given this construction, we can determine if M accepts w as follows:

1. Construct S as described above.
2. Run solver R on (S, x) for any input x .
3. If R accepts (S, x) , then S must move its head left three times in a row, which by our construction happens if and only if M accepts w .
4. If R rejects (S, x) , then S never moves its head left three times in a row, which means M does not accept w .

Since we have shown that we can reduce the problem of A_{TM} to the problem of "Given a Turing Machine M and input w , does M ever move its head to the left three times in a row?", and since A_{TM} is undecidable, we can conclude that this problem is also undecidable. $\square$